

Jenkins Task 2

Assignment Overview

This document covers six independent tasks: configuring two slave (agent) machines under the Jenkins master, configuring a GitHub webhook to trigger a Jenkins job on push, configuring Poll SCM and Build Periodically triggers on a job, taking a backup of the Jenkins server using a bash script, taking a backup of Jenkins using the ThinBackup plugin, and setting up a new Jenkins server and restoring the bash-script backup taken in Task 4. Each task resets and is documented with its own independent steps.

The following resource reference is used throughout this document:

Task	Resource	Purpose
1	Jenkins master + 2 agent nodes	Distribute build execution across dedicated agent machines
2	GitHub webhook -> Jenkins job	Trigger builds automatically on repository push events
3	Poll SCM / Build Periodically triggers	Trigger builds on a schedule, independent of webhooks
4	Bash backup script + JENKINS_HOME	Take a manual/scripted backup of the Jenkins server
5	ThinBackup plugin	Take a scheduled, UI-managed backup of Jenkins
6	New Jenkins server + Task 4 backup archive	Restore a Jenkins server from a bash-script backup

Task 1: Configure 2 Slave (Agent) Machines in the Jenkins Master

Outcome

Two additional machines are connected to the Jenkins master as agent nodes (agent-1 and agent-2), each showing as online with available executors, ready to run builds distributed away from the controller.

Objective

To extend Jenkins beyond the built-in controller node by attaching two dedicated agent machines, so build load can be distributed and jobs can be pinned to a specific environment via labels.

Prerequisites

- Jenkins master installed and reachable (Task 1 of the base install lab).
- Two Linux machines reachable from the master over SSH, each with Java (JDK 17 or 21) installed.
- A dedicated Jenkins SSH credential (username + private key, or username/password) valid on both agent machines.

Step-by-Step Implementation

Step 1: Prepare Both Agent Machines

```
sudo yum install java-21-amazon-corretto -y
```

Create a dedicated non-root user (e.g. jenkins-agent) on each machine and a remote root directory such as /home/jenkins-agent, used to store the agent workspace.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
ec2-user@ip-172-31-19-102 ~]$ sudo dnf install java-21-amazon-corretto -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
java-21-amazon-corretto	x86_64	1:21.0.12+8-1.amzn2023.1	amazonlinux	218 k
Installing dependencies:				
alsa-lib	x86_64	1.2.7.2-1.amzn2023.0.3	amazonlinux	503 k
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux	717 k
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.3 M
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	467 k
dejavu-serif-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.0 M
fontconfig	x86_64	2.13.94-2.amzn2023.0.2	amazonlinux	273 k
fonts-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux	9.5 k

```
Verifying      : libxcb-1.17.0-1.amzn2023.0.1.x86_64
Verifying      : pixman-0.43.4-1.amzn2023.0.4.x86_64
Verifying      : xml-common-0.6.3-56.amzn2023.0.2.noarch

Installed:
alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
glib-5.2.1-9.amzn2023.0.4.x86_64
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch
cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
fontconfig-2.13.94-2.amzn2023.0.2.x86_64
freetype-2.13.2-5.amzn2023.0.2.x86_64
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
javadoc-filesystem-6.0.0-7.amzn2023.0.6.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64

Complete!
```

```
root@ip-172-31-19-102 ec2-user]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
root@ip-172-31-19-102 ec2-user]#
```

```
root@ip-172-31-19-102 ec2-user]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:A37rk0noFNf5bVwJm7FQD4SMiYsHHrhneETD2epfE0c root@ip-172-31-19-102.ec2.internal
The key's randomart image is:
+---[RSA 3072]---+
|  += . + ++ |
|  =. + . E + oo |
|  = = o. . . *. |
|  o *. =. o + .. |
|  . o o+S . o |
|  . o+.o . + |
|  .o..oo . |
|  ...+ |
|  .. |
+----[SHA256]-----+
```

```
root@ip-172-31-19-102 ec2-user]# cd ~/.ssh
root@ip-172-31-19-102 .ssh]# ls
authorized_keys  id_rsa  id_rsa.pub
root@ip-172-31-19-102 .ssh]# cat id_rsa.pub > authorized_keys
root@ip-172-31-19-102 .ssh]# chmod 700 authorized_keys
root@ip-172-31-19-102 .ssh]#
```

Master Ec2:

```
[root@ip-172-31-23-63 ec2-user]# cat /var/lib/jenkins/secrets/initialAdminPassword
a034c9b4b7784cd7bc2b69063e2f9d7f
[root@ip-172-31-23-63 ec2-user]# ^C
[root@ip-172-31-23-63 ec2-user]# mkdir -p /var/lib/jenkins/.ssh
[root@ip-172-31-23-63 ec2-user]# cd /var/lib/jenkins/.ssh
[root@ip-172-31-23-63 .ssh]# ssh-keyscan -H 18.212.241.101 >>/var/lib/jenkins/.ssh/known_hosts
[root@ip-172-31-23-63 .ssh]# ll
total 4
-rw-r--r--. 1 root root 564 Aug 11 13:59 known_hosts
[root@ip-172-31-23-63 .ssh]# chown jenkins:jenkins known_hosts
[root@ip-172-31-23-63 .ssh]# ll
total 4
-rw-r--r--. 1 jenkins jenkins 564 Aug 11 13:59 known_hosts
[root@ip-172-31-23-63 .ssh]#
```

Step 2: Add Credentials for the Agents

Parameter	Value
Location	Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials
Kind	SSH Username with private key
ID	agent-ssh-key

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Credentials

[+ Add Credentials](#)

agent-ssh-key ec2-user

System - Global



Step 3: Create Node agent-1

Parameter	Value
Location	Manage Jenkins -> Nodes -> New Node
Node name	slave-01
Type	Permanent Agent
Remote root directory	/home/ec2-user/salve-workspace
Labels	java
Launch method	Launch agents via SSH
Host	IP/hostname of agent machine 1
Credentials	agent-ssh-key (created in Step 2)

 **Screenshot Evidence:** capture the console view for this step and insert it here.


Jenkins / Nodes ▾

🔍
 ⚙️
 👤

Name ?

Description ?

Plain text [Preview](#)

Number of executors ?

Remote root directory ?

Labels ?

Usage ?

Use this node as much as possible ▾

Launch method ?

Launch agents via SSH ▾ ?

Host ?

Credentials ?

▾
 + Add

Host Key Verification Strategy ?

▾ ?

Advanced ▾

Availability ?

Keep this agent online as much as possible ▾ ?

Step 4: Create Node agent-2

Parameter	Value
Node name	slave-02
Type	Permanent Agent
Remote root directory	/home/ec2-user/slave-workspace
Labels	python
Launch method	Launch agents via SSH

Parameter	Value
Host	IP/hostname of agent machine 2
Credentials	agent-ssh-key (created in Step 2)

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Name ?

slave-02

Description ?

Plain text [Preview](#)

Number of executors ?

2

Remote root directory ?

Remote root directory ?

/home/ec2-user/slave-workspace

Labels ?

python

Usage ?

Use this node as much as possible

Launch method ?

Launch agents via SSH

Host ?

107.20.27.112

107.20.27.112

Credentials ?

ec2-user

+ Add

Host Key Verification Strategy ?

Known hosts file Verification Strategy

Advanced

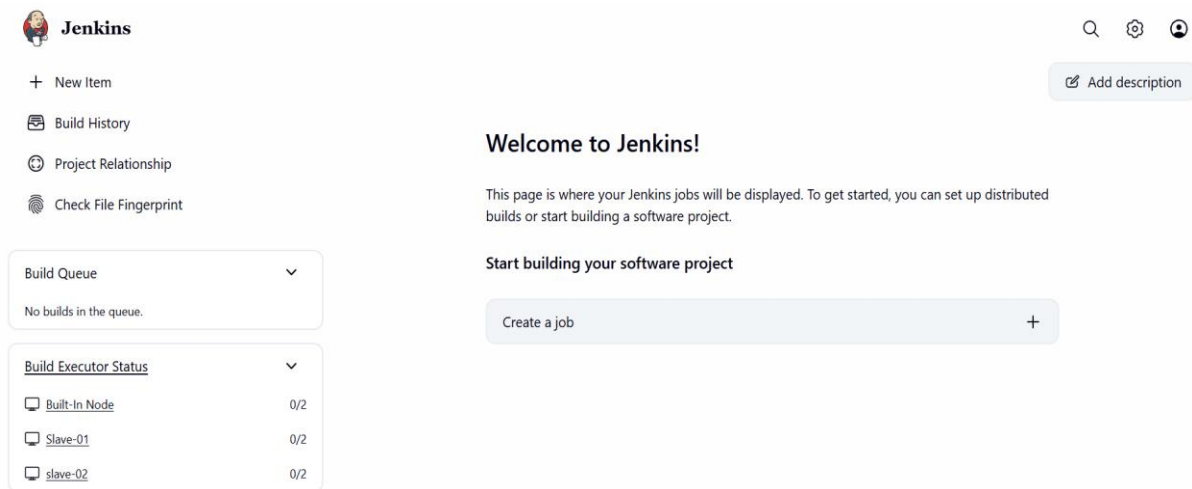
Availability ?

Keep this agent online as much as possible

Step 5: Verify Both Agents Connect

Open Manage Jenkins -> Nodes and confirm both agent-1 and agent-2 show a green status icon with the configured number of executors, instead of the red "disconnected" icon.

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Validation Steps

- The Nodes page lists agent-1 and agent-2 as online, each with at least one idle executor.
- The Log link for each node shows "Agent successfully connected and online" with no SSH errors.
- A test Freestyle job restricted to label linux-agent-1 builds on agent-1, and one restricted to linux-agent-2 builds on agent-2.

Explanation

Jenkins follows a master/agent architecture in which the controller schedules work and one or more agent nodes provide the executors that actually run builds. Connecting agent machines over SSH lets Jenkins launch and manage the agent process remotely, and assigning each node a distinct label makes it possible to route specific jobs to a specific machine rather than letting any job land on any executor.

Real-Time Use Case

Production Jenkins setups almost never build directly on the controller; teams attach agent pools split by operating system, toolchain, or team ownership so heavy or environment-specific builds do not compete with the controller for resources, and so a controller failure does not take build capacity down with it.

Common Errors and Troubleshooting

Error	Cause	Resolution
Agent stuck on "Launching agent"	SSH port 22 blocked or wrong host/IP entered	Verify firewall/security group allows SSH from the master, and re-check the host field
Agent shows offline immediately after connecting	Java not installed, or wrong Java version on agent	Install a supported JDK on the agent machine and reconnect the node
Authentication failed (SSH)	Wrong credential selected or key not authorized on agent	Re-create the credential and confirm the public key is in the agent user's authorized_keys

Task 2: Configure Webhooks to the Jenkins Job

Outcome

A GitHub webhook is configured against the target repository and the Jenkins job triggers automatically within seconds of a push, without any manual build or polling delay.

Objective

To replace manual or polling-based triggers with a push-based webhook, so Jenkins reacts to source changes immediately.

Prerequisites

- Task 1 (base install) completed and the job already has Git configured under Source Code Management.
- Jenkins reachable from GitHub over the internet (public URL, reverse proxy, or a tunneling tool such as ngrok).
- GitHub plugin installed (bundled with the suggested plugin set).

Step-by-Step Implementation

Step 1: Enable the GitHub Trigger on the Job

Parameter	Value
Location	Job -> Configure -> Build Triggers
Option	GitHub hook trigger for GITScm polling
State	Checked

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

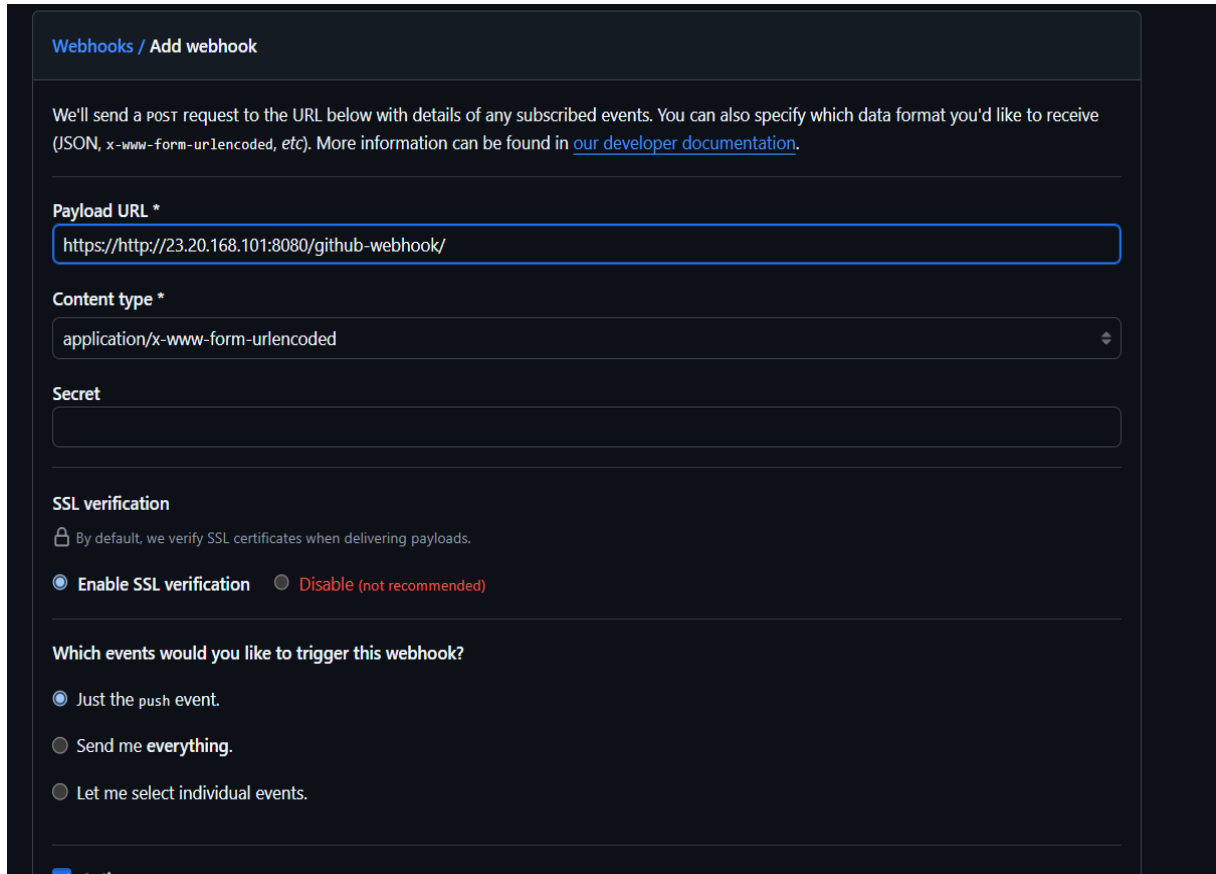
- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Step 2: Add the Webhook in GitHub

Parameter	Value
Location	GitHub repository -> Settings -> Webhooks -> Add webhook
Payload URL	http://<jenkins-url>/github-webhook/
Content type	application/json

Parameter	Value
Events	Just the push event
Active	Checked

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Webhooks / Add webhook

We'll send a `POST` request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, `x-www-form-urlencoded`, etc). More information can be found in [our developer documentation](#).

Payload URL *

`https://http://23.20.168.101:8080/github-webhook/`

Content type *

`application/x-www-form-urlencoded`

Secret

SSL verification

 By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

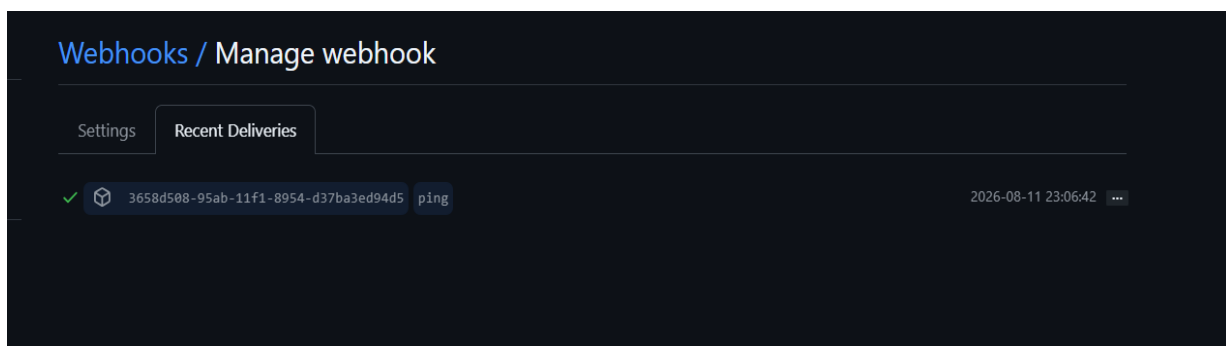
☐ Let me select individual events.

☒ Active

Step 3: Confirm Webhook Delivery in GitHub

Open the webhook's Recent Deliveries tab and confirm the initial ping and the most recent push both show a green checkmark with a 200 response code.

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Webhooks / Manage webhook

Settings Recent Deliveries

✓  3658d508-95ab-11f1-8954-d37ba3ed94d5 ping 2026-08-11 23:06:42 ...

Validation Steps

- A new build starts in Jenkins within a few seconds of the push, visible in Build History without clicking Build Now.
- The build's Console Output shows it was started by "GitHub push notification" rather than a manual or timer cause.

- GitHub's Recent Deliveries tab shows a 200 response for the push event that triggered the build.

Explanation

A webhook is an outbound HTTP call GitHub makes to Jenkins the moment a configured event happens. Because Jenkins reacts to that call directly, there is no polling interval to wait out; the GitHub plugin exposes a fixed endpoint (/github-webhook/) that receives the payload and starts any job whose trigger checkbox is enabled and whose repository URL matches.

Real-Time Use Case

Fast-feedback CI pipelines rely on webhook triggers so a broken build is reported back to a developer within moments of a push, rather than minutes later on the next scheduled poll, which materially shortens the fix-forward loop in an active team.

Common Errors and Troubleshooting

Error	Cause	Resolution
Webhook delivery shows 404	Payload URL missing the trailing /github-webhook/ path, or Jenkins URL misconfigured	Correct the payload URL and confirm Manage Jenkins -> System -> Jenkins URL matches the public address
Push does not trigger a build	GitHub hook trigger checkbox not enabled on the job	Open Job -> Configure -> Build Triggers and enable the GitHub hook trigger option
Webhook delivery times out / connection refused	Jenkins not reachable from the internet	Expose Jenkins via a reverse proxy or a tunneling tool, and re-test delivery from GitHub

Task 3: Configure Poll SCM and Build Periodically Options in the Jenkins Job

Outcome

The Jenkins job is configured with both a Poll SCM schedule and a separate Build Periodically schedule, and each behaves according to its own cron expression.

Objective

To trigger builds on a schedule without depending on a webhook, using Poll SCM to build only when the repository has changed and Build Periodically to build unconditionally on a fixed interval.

Prerequisites

- Task 1 (base install) completed and the job already has Git configured under Source Code Management.
- Basic familiarity with Jenkins cron syntax (MINUTE HOUR DOM MONTH DOW).

Step-by-Step Implementation

Step 1: Enable Poll SCM

Parameter	Value
Location	Job -> Configure -> Build Triggers
Option	Poll SCM

Parameter	Value
Schedule	H/5 * * * *
Behavior	Checks the repository every 5 minutes; builds only if new commits are found

 **Screenshot Evidence:** capture the console view for this step and insert it here.

☒ Poll SCM ?

Schedule ?

H/5 * * * *

Would last have run at Wednesday, August 12, 2026, 6:23:00 AM Coordinated Universal Time; would next run at Wednesday, August 12, 2026, 6:28:00 AM Coordinated Universal Time.

☐ Ignore post-commit hooks ?

Step 2: Enable Build Periodically

Parameter	Value
Location	Job -> Configure -> Build Triggers
Option	Build periodically
Schedule	H 0 * * *
Behavior	Builds once every day regardless of whether the repository changed

 **Screenshot Evidence:** capture the console view for this step and insert it here.

☒ Build periodically ?

Schedule ?

H 0 * * *

Would last have run at Wednesday, August 12, 2026, 12:28:00 AM Coordinated Universal Time; would next run at Thursday, August 13, 2026, 12:28:00 AM Coordinated Universal Time.

☐ GitHub hook trigger for GITScm polling ?
☐ Poll SCM ?

Step 3: Save and Inspect Scheduled Trigger Times

Open the job's Poll Log (for Poll SCM) and the job status page (for the next scheduled build countdown) to confirm both triggers registered a next-run time.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

	#10	36 sec	0.16 sec	Built-In Node	
	#9	1 min 36 sec	0.16 sec	Built-In Node	
	#8	2 min 36 sec	0.19 sec	Built-In Node	
	#7	15 min	3.7 sec	Built-In Node	

Validation Steps

- The job's Git Polling Log shows periodic entries such as "Polling SCM" with either "No changes" or "Changes found" every 5 minutes.
- Build History shows a build starting daily at the scheduled hour even with no repository changes, caused by "Started by timer".
- A commit pushed between polling cycles results in a new build within one polling interval.

Explanation

Poll SCM and Build Periodically both use the same cron-style schedule syntax but serve different purposes: Poll SCM asks the source control system whether anything changed and only starts a build when it has, while Build Periodically ignores source control state entirely and simply runs on the clock, which makes it suited to routine builds, reports, or housekeeping jobs that must run regardless of code changes.

Real-Time Use Case

Teams without webhook access, such as an internal Jenkins instance that cannot be reached from a SaaS Git host, commonly fall back to Poll SCM as the trigger mechanism, while Build Periodically is kept for nightly full builds, dependency-update checks, or scheduled report jobs that are not tied to a specific commit.

Common Errors and Troubleshooting

Error	Cause	Resolution
Trigger not saved / schedule ignored	Invalid cron syntax entered	Use the Jenkins schedule validator hint shown under the field and correct the five-field expression
Poll Log always shows "No changes"	Branch specifier in Git config does not match the branch receiving commits	Correct the branch specifier under Source Code Management to match the active branch
Build Periodically never fires at the expected time	Jenkins server time zone differs from the expected local time	Check Manage Jenkins -> System time, or set the TZ environment for the schedule explicitly

Task 4: Take a Backup of the Jenkins Server Using a Bash Script

Outcome

A bash script exists on the Jenkins server that archives the entire JENKINS_HOME directory into a timestamped tar.gz file, producing a portable backup that can be copied off the server or restored later.

Objective

To create a lightweight, scriptable backup mechanism that captures Jenkins configuration, job definitions, build history, plugins, and credentials without relying on a plugin.

Prerequisites

- Shell (SSH) access to the Jenkins server with sudo privileges.
- Sufficient free disk space to hold a compressed copy of JENKINS_HOME.
- JENKINS_HOME location confirmed (default /var/lib/jenkins on RPM/Debian installs).

Step-by-Step Implementation

Step 1: Create the Backup Script

Parameter	Value
Script path	/usr/local/bin/Jenkins_backup.sh
Source directory	/var/lib/jenkins
Destination directory	/opt/jenkins-backups
Archive name pattern	jenkins-backup-<YYYY-MM-DD_HH-MM>.tar.gz

```
sudo vi /usr/local/bin/jenkins_backup.sh
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
#!/bin/bash

# Jenkins home directory
JENKINS_HOME="/var/lib/jenkins"

# Backup directory
BACKUP_DIR="/backup/jenkins"

# Current date and time
DATE=$(date +%Y-%m-%d_%H-%M-%S)

# Backup file name
BACKUP_FILE="$BACKUP_DIR/jenkins_backup_$DATE.tar.gz"

# Create backup directory if it doesn't exist
mkdir -p "$BACKUP_DIR"

echo "Starting Jenkins backup..."

# Create compressed backup
tar -czf "$BACKUP_FILE" "$JENKINS_HOME"

# Check whether backup was successful
if [ $? -eq 0 ]; then
    echo "Jenkins backup completed successfully."
    echo "Backup file: $BACKUP_FILE"
else
    echo "Jenkins backup failed."
    exit 1
fi

# Delete backups older than 7 days
find "$BACKUP_DIR" -type f -name "jenkins_backup_*.tar.gz" -mtime +7 -delete

echo "Old backups deleted."
echo "Backup process completed."
~
~
~
~
```

Step 2: Make the Script Executable and Run It

`sudo chmod +x /usr/local/bin/jenkins_backup.sh`

`sudo /usr/local/bin/jenkins_backup.sh`

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[root@ip-172-31-23-63 ec2-user]# sudo chmod +x /usr/local/bin/jenkins_backup.sh
[root@ip-172-31-23-63 ec2-user]# sudo /usr/local/bin/jenkins_backup.sh
Starting Jenkins backup...
tar: Removing leading '/' from member names
Jenkins backup completed successfully.
Backup file: /backup/jenkins/jenkins_backup_2026-08-12_07-04-55.tar.gz
Old backups deleted.
Backup process completed.
[root@ip-172-31-23-63 ec2-user]# sudo ls -lh /backup/jenkins/
total 144M
-rw-r--r--. 1 root root 144M Aug 12 07:05 jenkins_backup_2026-08-12_07-04-55.tar.gz
[root@ip-172-31-23-63 ec2-user]#
```

Validation Steps

- `ls -lh /opt/jenkins-backups` shows a `jenkins-backup-<timestamp>.tar.gz` file with a recent modification time and non-zero size.
- `tar tzf` on the archive lists `config.xml`, `jobs/`, `users/`, and `secrets/` from `JENKINS_HOME`.
- The script exits with status 0 and prints the confirmation message with the backup path.

Explanation

JENKINS_HOME is a self-contained directory holding every job definition, build record, plugin, user account, and encrypted credential, so archiving it in full is functionally equivalent to snapshotting the entire Jenkins server. Excluding the workspace and war subdirectories keeps the archive smaller, since those directories are regenerated automatically and are not required to restore configuration or history.

Real-Time Use Case

Teams that do not want a plugin dependency, or that need backups feeding into an existing external backup/cron pipeline (e.g. syncing to S3 or a NAS), commonly script the backup this way so it fits alongside backups already taken for other servers.

Common Errors and Troubleshooting

Error	Cause	Resolution
Permission denied writing to JENKINS_HOME	Script run as a user without read access to Jenkins files	Run the script with sudo or as the jenkins user, which owns JENKINS_HOME
tar fails with "No space left on device"	Backup destination disk is full	Free space or point BACKUP_DIR at a volume with sufficient capacity
Archive missing recent jobs	Script executed while a build was actively writing to JENKINS_HOME	Re-run the backup when no builds are in progress, or stop the Jenkins service before archiving for a fully consistent snapshot

Task 5: Take a Backup of Jenkins Using the ThinBackup Plugin

Outcome

The ThinBackup plugin is installed and configured with a scheduled full backup, and a manual backup run produces a timestamped backup set under the configured backup directory.

Objective

To use a UI-managed, scheduled backup mechanism with built-in restore support, as an alternative to a hand-written script.

Prerequisites

- Task 1 (base install) completed and administrator access to Jenkins.
- A backup directory on disk that the Jenkins service account can write to.

Step-by-Step Implementation

Step 1: Install the ThinBackup Plugin

Manage Jenkins -> Plugins -> Available plugins -> search "ThinBackup" -> Install without restart.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings
- Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

ThinBackup

 Success

Loading plugin extensions

 Success

→ [Go back to the top page](#)

(you can start using the installed plugins right away)

→

Restart Jenkins when installation is complete and no jobs are running

Step 2: Configure ThinBackup Settings

Parameter	Value
Location	Manage Jenkins -> ThinBackup -> Settings
Backup directory	/opt/jenkins-thinbackup
Full backup schedule	0 1 * * 0 (weekly, Sunday 01:00)
Diff backup schedule	0 1 * * 1-6 (daily except Sunday)
Max number of backup sets to keep	10
Backup build results	Enabled

 Screenshot Evidence: capture the console view for this step and insert it here.

ThinBackup Configuration

Backup directory ?

/opt/jenkins-thinbackup

Backup path must not be empty.

Backup schedule for full backups ?

H 1 * * 0

Backup schedule for differential backups ?

0 1 * * 1-6

Max number of backup sets ?

10

Files excluded from backup (regular expression) ?

Files excluded from backup (regular expression) ?

☒ Wait until Jenkins is idle to perform a backup ?

Force Jenkins to quiet mode after specified minutes ?

120

☒ Backup build results ?

☐ Backup build archive ?

☒ Backup only builds marked to keep ?

☐ Backup userContent folder ?

☐ Backup next build number file ?

☐ Backup plugins archives ?

☐ Backup config-history folder ?

☐ Backup additional files ?

☐ Clean up differential backups ?

☐ Move old backups to ZIP files ?

☒ Stop the backup as soon as an exception occurs in the file handling ?

Step 3: Trigger a Manual Backup

Manage Jenkins -> ThinBackup -> Backup Now, and wait for the confirmation message that the backup completed.

Step 4: Verify the Backup Set

ls /opt/jenkins-thinbackup

Confirm a folder named FULL-<yyyy-MM-dd_HH-mm> was created containing config.xml, jobs/, and users/.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[root@ip-172-31-23-63 ec2-user]# sudo mkdir -p /opt/jenkins-thinbackup
[root@ip-172-31-23-63 ec2-user]# sudo chown -R jenkins:jenkins /opt/jenkins-thinbackup
[root@ip-172-31-23-63 ec2-user]# sudo chmod 750 /opt/jenkins-thinbackup
[root@ip-172-31-23-63 ec2-user]# sudo -u jenkins touch /opt/jenkins-thinbackup/test
[root@ip-172-31-23-63 ec2-user]# ls -l /opt/jenkins-thinbackup/
total 0
-rw-r--r--. 1 jenkins jenkins 0 Aug 12 07:23 test
[root@ip-172-31-23-63 ec2-user]# sudo rm /opt/jenkins-thinbackup/test
[root@ip-172-31-23-63 ec2-user]# sudo ls -lah /opt/jenkins-thinbackup
total 16K
drwxr-x---. 3 jenkins jenkins 35 Aug 12 07:25 .
drwxr-xr-x. 4 root root 43 Aug 12 07:22 ..
drwxr-xr-x. 5 jenkins jenkins 16K Aug 12 07:25 FULL-2026-08-12_07-25
[root@ip-172-31-23-63 ec2-user]#
```

Validation Steps

- The ThinBackup Settings page shows the configured schedules saved without validation errors.
- The backup directory contains a FULL-<timestamp> folder after running Backup Now.
- Manage Jenkins -> ThinBackup -> Restore lists the new backup set as an available restore point.

Explanation

ThinBackup automates what the bash-script approach does manually: it archives JENKINS_HOME on a schedule, distinguishes between full backups (a complete snapshot) and differential backups (only what changed since the last full backup), and enforces a retention limit so old backup sets are pruned automatically instead of accumulating indefinitely.

Real-Time Use Case

Operations teams that want backup and restore handled without maintaining a custom script, and who want a visible restore point selector inside Jenkins itself, typically standardize on ThinBackup (or an equivalent plugin) for its scheduling and retention management rather than building that logic by hand.

Common Errors and Troubleshooting

Error	Cause	Resolution
Backup directory not writable	Jenkins service account lacks permission on the target path	chown the directory to the jenkins user, or choose a path already writable by that account
No backups appear after the schedule time passes	Cron expression invalid or plugin not reloaded after install	Re-open ThinBackup Settings, correct the schedule, and save again to re-register the trigger
Backup directory grows without bound	Max number of backup sets left at its default/unset	Set a retention limit under ThinBackup Settings so older backup sets are deleted automatically

Task 6: Set Up a New Jenkins Server and Restore the Backup Taken in Task 4

Outcome

A freshly installed Jenkins server is restored from the bash-script backup archive created in Task 4, and the new server shows the same jobs, users, and build history as the original.

Objective

To validate that the Task 4 backup archive is a complete, restorable snapshot by recovering it onto an independent Jenkins installation, confirming the backup strategy works end to end.

Prerequisites

- The Jenkins_backup-<timestamp>.tar.gz archive produced in Task 4, copied to the new server.
- A new Linux server with Jenkins freshly installed (same major version recommended) but not yet unlocked/configured.
- Sudo access on the new server.

Step-by-Step Implementation

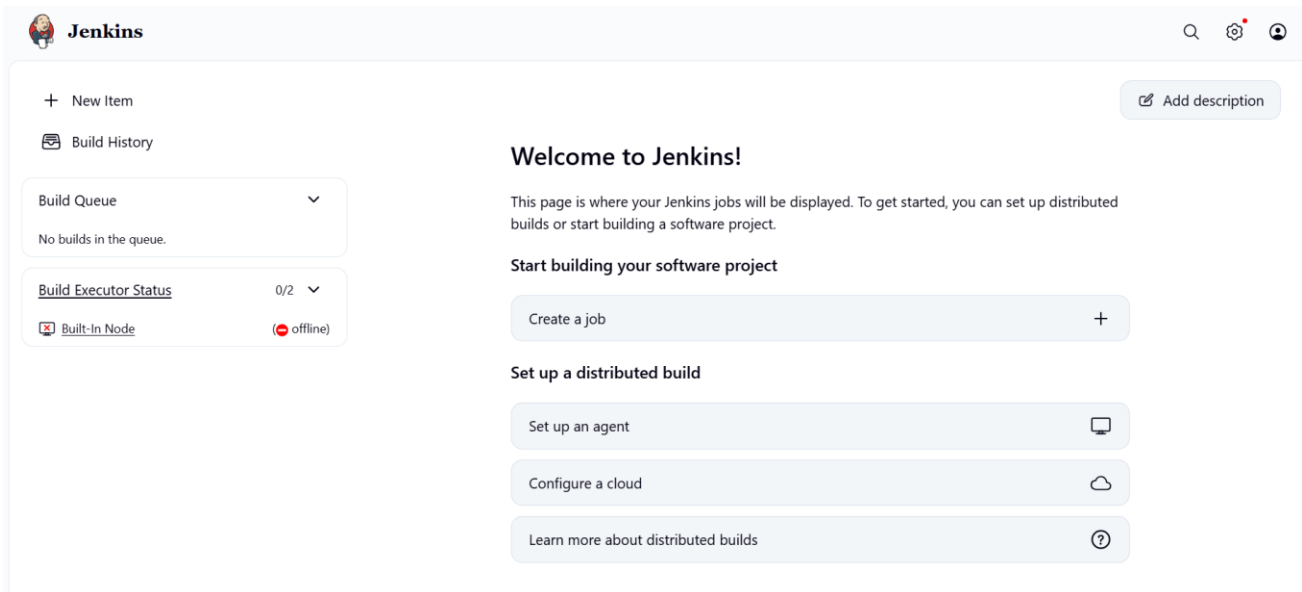
Step 1: Install Jenkins on the New Server and Stop the Service

```
sudo yum install jenkins -y
```

```
sudo systemctl stop jenkins
```

Stopping the service before restoring prevents Jenkins from writing its own default JENKINS_HOME while the archive is being extracted.

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Step 2: Copy the Task 4 Backup Archive to the New Server

`scp jenkins-backup-2026-08-12_07-04-55.tar.gz user@new-server:/tmp/`

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[root@ip-172-31-23-63 jenkins]# vi jenkins.pem
[root@ip-172-31-23-63 jenkins]# chmod 600 jenkins.pem
[root@ip-172-31-23-63 jenkins]# ll
total 146916
-rw-----. 1 root root    1675 Aug 12 10:15 jenkins.pem
-rw-r--r--. 1 root root 150434695 Aug 12 07:05 jenkins_backup_2026-08-12_07-04-55.tar.gz
[root@ip-172-31-23-63 jenkins]# scp -i jenkins.pem jenkins_backup_2026-08-12_07-04-55.tar.gz ec2-user@ec2-52-4-99-82.compute-1.amazonaws.com:/tmp/
jenkins_backup_2026-08-12_07-04-55.tar.gz                                100% 143MB 226.8MB/s   00:00
[root@ip-172-31-23-63 jenkins]#
```

Step 3: Extract the Archive into JENKINS_HOME

`sudo rm -rf /var/lib/jenkins/*`

`sudo tar xzf /tmp/jenkins-backup-2026-08-05_02-00.tar.gz -C /var/lib/jenkins --strip-components=1`

`sudo chown -R jenkins:jenkins /var/lib/jenkins`

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[ec2-user@ip-172-31-19-102 ~]$ cd /tmp
[ec2-user@ip-172-31-19-102 tmp]$ ll
total 146912
drwxr-xr-x. 2 ec2-user ec2-user    40 Aug 12 09:56 hspanferdata_ec2-user
drwxr-xr-x. 2 jenkins jenkins    40 Aug 12 10:09 hspanferdata_jenkins
-rw-r--r--. 1 ec2-user ec2-user 150434695 Aug 12 10:16 jenkins_backup_2026-08-12_07-04-55.tar.gz
drwx-----. 3 root root    60 Aug 12 06:12 systemd-private-3201ad4b33974d39a0074e7133b0492f-chrond.service-18eAnc
drwx-----. 3 root root    60 Aug 12 06:12 systemd-private-3201ad4b33974d39a0074e7133b0492f-dbus-broker.service-thDr1p
drwx-----. 3 root root    60 Aug 12 06:12 systemd-private-3201ad4b33974d39a0074e7133b0492f-policy-routes@ens5.service-15831y
drwx-----. 3 root root    60 Aug 12 06:12 systemd-private-3201ad4b33974d39a0074e7133b0492f-systemd-logind.service-LCO07V
drwx-----. 3 root root    60 Aug 12 06:12 systemd-private-3201ad4b33974d39a0074e7133b0492f-systemd-resolved.service-cAVJUe
[ec2-user@ip-172-31-19-102 tmp]$ sudo tar xzf /tmp/jenkins_backup_2026-08-12_07-04-55.tar.gz -C /var/lib/jenkins --strip-components=1
[ec2-user@ip-172-31-19-102 tmp]$ sudo chown -R jenkins:jenkins /var/lib/jenkins
[ec2-user@ip-172-31-19-102 tmp]$
```

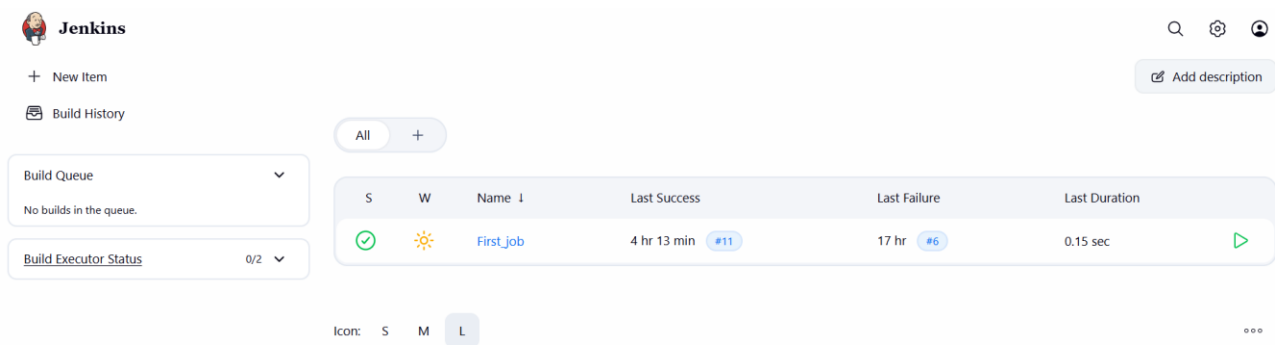
Step 4: Start Jenkins and Confirm the Restore

`sudo systemctl start jenkins`

`sudo systemctl status jenkins`

Log in to `http://<new-server-ip>:8080` (or 8081 if configured that way) using the original administrator credentials restored from the archive, and confirm the setup wizard does not reappear.

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Validation Steps

- All jobs that existed on the original server (including the agent nodes, webhook-triggered job, and scheduled jobs from Tasks 1-3) are visible on the new server without being recreated.
- Existing users (DevOps, Testing, or any created earlier) can still log in with their original credentials.
- Build History for restored jobs still contains the original build numbers and console logs rather than starting over at #1.

Explanation

Because the Task 4 script backs up the entire JENKINS_HOME directory rather than individual settings, restoring is simply a matter of placing that same directory structure on a new server and letting Jenkins start against it; every job, credential, and plugin reference is read back exactly as it was captured, with no manual reconfiguration required beyond fixing file ownership.

Real-Time Use Case

This is the same procedure used for real disaster-recovery drills, server migrations (e.g. moving Jenkins to a larger instance or a new OS version), and Jenkins version upgrades performed by standing up a clean server rather than upgrading in place.

Common Errors and Troubleshooting

Error	Cause	Resolution
Jenkins fails to start after restore	Restored files owned by the wrong user	Run <code>chown -R jenkins:jenkins</code> on JENKINS_HOME and restart the service
Setup wizard reappears instead of restored dashboard	secrets/ directory was excluded or not fully extracted	Re-extract the archive ensuring the secrets/ folder is present, since it holds the unlock/install state
Plugins missing or jobs fail to load	Archive extracted over a JENKINS_HOME that still had default files mixed in	Clear the target JENKINS_HOME completely before extracting, then re-run the restore